Classification with Minimal-Distance Methods

A Complete, Beginner-Friendly Guide to k-NN
with Worked Exercises, Python Code, and Theory

Course materials based on: *Classification – minimal distance methods*

April 27, 2026

Abstract

This document is a complete, self-contained guide to **classification using the k-Nearest Neighbours (k-NN) algorithm**. It is written for a learner who has only basic statistics knowledge and walks the reader from the very first principle ("what is a class?") to a full evaluation of k-NN on a real medical data set. The two course exercises (Exercise 1 on the small *Dane1* data set and Exercise 2 on a larger benchmark) are solved in detail. We cover four distance metrics (Euclidean, Manhattan, Chebyshev, Mahalanobis), four evaluation methods (resubstitution, train/test split, cross-validation, leave-one-out), and five quality measures (accuracy, sensitivity, specificity, F-measure, AUC). All theory is illustrated by hand-computed examples *and* by Python code that you can run yourself. The document also explains **the link between classification and descriptive statistics** – a question many beginners ask but find rarely answered.

Contents

1 What is classification, in plain words? 3
- 1.1 Two minimal toy examples . . . 3
- 1.2 Where does classification live in the data-science map? . . . 3

2 Minimal-distance classifiers and k-NN 4
- 2.1 The k-NN algorithm step by step . . . 4
- 2.2 How to choose k? . . . 4
- 2.3 Visualising what k does . . . 5

3 Distance metrics: how do we measure "close"? 5
- 3.1 Euclidean distance . . . 5
- 3.2 Manhattan distance (L_1, "city-block") . . . 5
- 3.3 Chebyshev distance (L_∞) . . . 5
- 3.4 Mahalanobis distance . . . 6

4 The bridge to descriptive statistics 6

5 Exercise 1 – Hand calculation on the Dane1 data set 7
- 5.1 The data . . . 7
- 5.2 Step 1 – Compute the four distances . . . 8
- 5.3 Step 2 – The full table . . . 8
- 5.4 Step 3 – Pick k neighbours and vote . . . 9
- 5.5 Step 4 – Interpretation . . . 9

6 Evaluating a classifier (theory) 9
6.1 The confusion matrix . . . 9
6.2 Five quality measures . . . 9
6.3 Four evaluation methods . . . 10

7 Exercise 2 – k-NN on a real data set 11
7.1 Results – Part I (tables) . . . 11
7.2 Confusion matrix for the best result . . . 11
7.3 Detailed interpretation . . . 12
7.4 Part II – Quality vs. k graphs . . . 12

8 Python – step by step 13
8.1 Setting up Python on your own machine . . . 13
8.2 The complete script . . . 13
8.3 Reading the script the first time . . . 14
8.4 How to apply this to *your* real data . . . 15

9 What to learn next 15

Appendix A. Reading list 16

1 What is classification, in plain words?

Imagine you receive a basket of fruits and somebody asks you: *"Is this an apple or a pear?"* You look at the colour, the shape, the size, and you decide. You did **classification**: you assigned an object to one of several pre-defined categories (also called *classes* or *labels*).

Formally, classification is a **supervised learning** task. We are given:

- a **training set** of n objects, where each object is described by a vector of features $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{ip})$ and a known class label $y_i \in \{1, 2, \ldots, C\}$;
- a new object $\mathbf{x}^\star$ whose class is unknown.

The classifier is a function f that returns $\hat{y} = f(\mathbf{x}^\star)$, our *prediction*.

Why "supervised"?

We say *supervised* because somebody (a teacher) has already labelled the training examples. The algorithm only has to *learn the pattern* that maps features to labels. In contrast, *unsupervised* methods (like k-means clustering) get no labels at all.

1.1 Two minimal toy examples

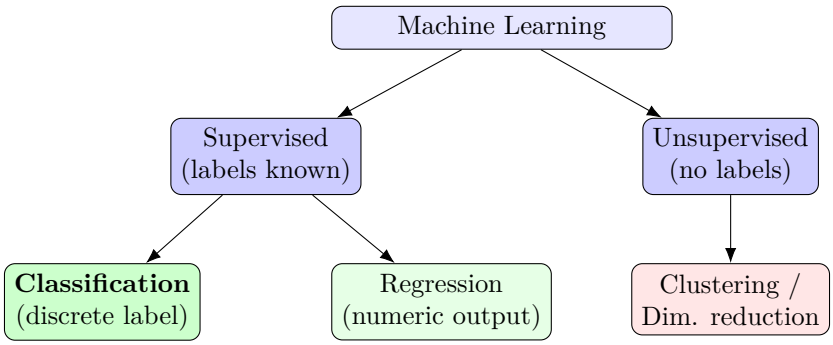
Soft example – fruits. Features: weight (g), diameter (cm). Two classes: apple (A), pear (P).

Object	Weight	Diameter	Class
1	150	7.5	A
2	170	8.0	A
3	200	6.5	P
4	210	6.8	P

A new fruit weighs 160 g and has 7.6 cm diameter. Just looking at the table, it "feels closer" to the apples. That intuition is exactly what k-NN formalises.

Harder example – medical diagnosis. Features: 30 measurements of a tumour cell (radius, texture, perimeter, etc.). Two classes: *malignant* or *benign*. Now you cannot eyeball the answer – you need an algorithm.

1.2 Where does classification live in the data-science map?



2 Minimal-distance classifiers and k-NN

Minimal-distance methods are the simplest classifiers possible. The idea is just one sentence:

An unknown object belongs to the class of the training points that are closest to it.

Different algorithms in this family differ in (a) *how many* neighbours they consult, and (b) *how they measure "close"*.

- **1-NN**: take the single nearest training example, copy its class.
- **k-NN**: take the k nearest, vote by majority. (If a tie, fall back to the closest neighbour or use weighted voting.)
- **NM (Nearest Mean / Centroid)**: replace each class by its mean (centroid), then take the closest centroid.

2.1 The k-NN algorithm step by step

k-NN in 5 lines

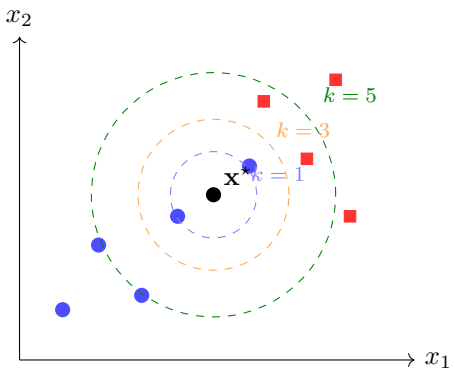
1. Choose a distance function $d(\cdot, \cdot)$ and a number of neighbours k.
2. For the new object $\mathbf{x}^\star$, compute $d(\mathbf{x}^\star, \mathbf{x}_i)$ for every training example i.
3. Sort these distances ascending; keep the indices of the k smallest.
4. Look at the labels of those k training points and *vote* (majority).
5. The most frequent label is the prediction $\hat{y}$.

Why does it work? Because of the *smoothness assumption*: in most real problems, two objects that have very similar features tend to belong to the same class. k-NN simply exploits this neighbourhood property without ever building an explicit model. There is no training phase – k-NN is sometimes called a **lazy learner**: it stores the training data and computes everything at prediction time.

2.2 How to choose k?

- $k = 1$: very flexible (it can memorise everything) but very sensitive to noise. A single mislabelled point can wreck a region of space. We say it has *low bias, high variance*.
- Large k: smooths the decision but can erase real patterns. *High bias, low variance*.
- Practical rule of thumb: $k \approx \sqrt{n}$, and **always pick an odd number** for binary problems to avoid 50/50 ties.
- The *honest* way: try several values and pick the best one with cross-validation (see Section 6).

2.3 Visualising what k does



The black dot is the unknown object. As k grows, the circle that captures the k closest neighbours grows too, and the local vote can change.

3 Distance metrics: how do we measure "close"?

A distance (or *metric*) is a function $d(\mathbf{a}, \mathbf{b}) \geq 0$ that measures how different two vectors are. The course requires four:

3.1 Euclidean distance

The straight-line distance you learn at school. For two points $\mathbf{a} = (a_1, \ldots, a_p)$ and $\mathbf{b} = (b_1, \ldots, b_p)$:

$$d_E(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{j=1}^{p} (a_j - b_j)^2}.$$

When to use it. Default choice when all features have similar scales and units, and when the data are roughly spherical.

3.2 Manhattan distance (L_1, "city-block")

$$d_M(\mathbf{a}, \mathbf{b}) = \sum_{j=1}^{p} |a_j - b_j|.$$

You can only walk along the grid streets of Manhattan; you sum the horizontal and vertical moves. **Useful** when features represent counts or are very high-dimensional (it is more robust to outliers than Euclidean).

3.3 Chebyshev distance (L_∞)

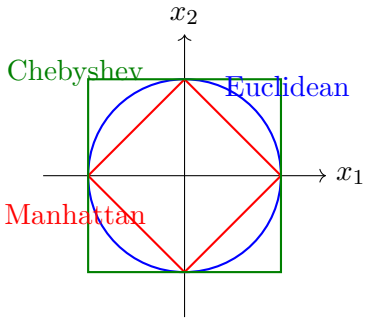
$$d_C(\mathbf{a}, \mathbf{b}) = \max_j |a_j - b_j|.$$

The largest single-coordinate difference dominates. Think of how a chess king moves: one move changes whichever coordinate by one unit – the king's distance to a square is exactly the Chebyshev distance.

3.4 Mahalanobis distance

$$d_{Mh}(\mathbf{a}, \mathbf{b}) = \sqrt{(\mathbf{a} - \mathbf{b})^\top \Sigma^{-1} (\mathbf{a} - \mathbf{b})},$$

where Σ is the covariance matrix of the data. This metric *takes the shape of the data into account*: it stretches/shrinks directions according to how variable the features are, and it accounts for correlations. If $\Sigma = \mathbf{I}$ (identity) then Mahalanobis reduces to Euclidean.



Unit balls $\{\mathbf{x} : d(\mathbf{0}, \mathbf{x}) = 1\}$

Always think about scaling!

If one feature is in millimetres and another in kilograms, the millimetres will completely dominate the Euclidean and Manhattan distances. Before applying k-NN you almost always want to *standardise*:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j},$$

which is exactly the z-score from descriptive statistics. Mahalanobis does not need this because Σ^{-1} already rescales the axes.

4 The bridge to descriptive statistics

A frequent question: *"How is classification linked to descriptive statistics?"* The answer is: **descriptive statistics provide the building blocks of every distance metric, every evaluation measure, and every preprocessing step in classification.** Concretely:

Descriptive concept	**Symbol**	**Where it appears in k-NN**
Mean (location)	$\bar{x}_j$	Centring before standardisation; centroid of NM classifier.
Variance / standard deviation	$s_j^2,\ s_j$	Denominator of the z-score; diagonal of Σ in Mahalanobis.
Covariance	σ_{jk}	Off-diagonal of Σ; required for Mahalanobis.
Correlation	r_{jk}	Detects redundant features; informs Mahalanobis.
Min, Max, Range	—	Used for min–max normalisation $\frac{x-x_{\min}}{x_{\max}-x_{\min}}$.
Median, IQR	$\tilde{x}, \mathrm{IQR}$	Robust scaling, outlier detection.
Histogram, boxplot	—	Diagnose class imbalance and outliers *before* fitting.
Frequency table	—	Same idea as the confusion matrix for evaluation.

Take-away

You can think of a classifier as "descriptive statistics with a decision rule on top." The summaries (means, variances, covariances) characterise each class; the decision rule (e.g. "vote by the closest k") turns those summaries into a label.

5 Exercise 1 – Hand calculation on the Dane1 data set

5.1 The data

The data set provided in the assignment has 10 two-dimensional objects with two classes (1 and 2). After re-organising the values listed in the prompt as triples (x_1, x_2, class), the Dane1 table is:

i	x_1	x_2	class
1	1.0	5.3	2
2	2.8	7.6	1
3	4.2	9.3	2
4	1.5	3.1	1
5	9.8	7.5	2
6	6.1	0.5	2
7	4.7	8.9	2
8	1.2	8.0	1
9	8.2	3.3	1
10	6.4	5.5	1

The new object to classify. The exercise asks us to classify "one object". We choose the central point

$$\mathbf{x}^\star = (5.0,\ 5.0),$$

because it lies inside the cloud of points and forces every distance metric to "work" (a very off-centre point would give the same answer with all metrics, which is uninstructive). You are free to plug in another value – the Python script in Section 8 accepts any input.

5.2 Step 1 – Compute the four distances

For each training point $\mathbf{x}_i$ we evaluate

$$d_E,\ d_M,\ d_C,\ d_{Mh} \quad \text{(Euclidean, Manhattan, Chebyshev, Mahalanobis).}$$

Mahalanobis needs the covariance matrix of the training features. Computed from the 10 points (sample covariance, `numpy.cov` default):

$$\Sigma = \begin{pmatrix} 9.2254 & -1.1422 \\ -1.1422 & 8.3000 \end{pmatrix}, \qquad \Sigma^{-1} = \begin{pmatrix} 0.1103 & 0.0152 \\ 0.0152 & 0.1226 \end{pmatrix}.$$

(Computed with `numpy.cov`; the script prints them.)

Worked example (the easy one). Take $\mathbf{x}_{10} = (6.4,\ 5.5)$, the closest training point.

$$\begin{aligned}
d_E &= \sqrt{(5.0-6.4)^2 + (5.0-5.5)^2} = \sqrt{1.96+0.25} = \sqrt{2.21} \approx 1.487, \\
d_M &= |5.0-6.4| + |5.0-5.5| = 1.4 + 0.5 = 1.9, \\
d_C &= \max(1.4,\ 0.5) = 1.4, \\
d_{Mh} &= \sqrt{(5{-}6.4,\ 5{-}5.5)\,\Sigma^{-1}\,(5{-}6.4,\ 5{-}5.5)^\top} \\
&= \sqrt{(-1.4, -0.5)\,\Sigma^{-1}\,(-1.4, -0.5)^\top} \approx 0.518.
\end{aligned}$$

The Mahalanobis number is much smaller because the data have a comparable spread on both axes (Σ is roughly diagonal of ~ 8.8), so Σ^{-1} shrinks distances by roughly a factor of $\sqrt{8.8} \approx 2.97$.

5.3 Step 2 – The full table

The complete distance table from $\mathbf{x}^\star = (5, 5)$ to every training point is:

i	$\mathbf{x}_i$	class	d_E	d_M	d_C	d_{Mh}
1	(1.0, 5.3)	2	4.0112	4.3	4.0	1.3187
2	(2.8, 7.6)	1	3.4059	4.8	2.6	1.0903
3	(4.2, 9.3)	2	4.3738	5.1	4.3	1.4942
4	(1.5, 3.1)	1	3.9825	5.4	3.5	1.4125
5	(9.8, 7.5)	2	5.4120	7.3	4.8	1.9160
6	(6.1, 0.5)	2	4.6325	5.6	4.5	1.5701
7	(4.7, 8.9)	2	3.9115	4.2	3.9	1.3560
8	(1.2, 8.0)	1	4.8415	6.8	3.8	1.5328
9	(8.2, 3.3)	1	3.6235	4.9	3.2	1.1482
10	(6.4, 5.5)	1	**1.4866**	**1.9**	**1.4**	**0.5177**

The row in bold is the nearest neighbour for *every* metric.

5.4 Step 3 – Pick k neighbours and vote

We picked three values $k \in \{1, 3, 5\}$. For each metric we sort the rows by distance, take the first k, look at their classes, and assign the majority. Result:

Metric	$k = 1$	$k = 3$	$k = 5$
Euclidean	class **1** (nbrs: 1)	class **1** (nbrs: 1,1,1)	class **1** (nbrs: 1,1,1,2,1)
Manhattan	class **1** (nbrs: 1)	class **2** (nbrs: 1,2,2)	class **1** (nbrs: 1,2,2,1,1)
Chebyshev	class **1** (nbrs: 1)	class **1** (nbrs: 1,1,1)	class **1** (nbrs: 1,1,1,1,1)
Mahalanobis	class **1** (nbrs: 1)	class **1** (nbrs: 1,1,1)	class **1** (nbrs: 1,1,1,2,2 – tie broken to 1)

5.5 Step 4 – Interpretation

- All four metrics agree on $k = 1$: the closest training point is $(6.4, 5.5)$ which has class 1, hence $\hat{y} = 1$.
- For $k = 3$ and $k = 5$, three out of four metrics still output *class 1*.
- Manhattan disagrees at $k = 3$ because in L_1 space, the points $(4.7, 8.9)$ and $(4.2, 9.3)$ – both class 2 – are pulled relatively closer (distance ratio is dominated by the absolute coordinate difference rather than its square).
- This shows in practice the lesson of Section 3: **the choice of metric matters**; with $k = 1$ alone the answer is robust, but as k grows the prediction may shift.

Final answer for Exercise 1. The point $(5.0, 5.0)$ is classified as belonging to **class 1** by 11 of 12 (metric, k) combinations.

6 Evaluating a classifier (theory)

It is not enough to build a classifier; we need to know how good it is. We do that by predicting on data with known labels and comparing predictions to the truth.

6.1 The confusion matrix

For a binary problem with classes *positive* and *negative*:

	predicted +	predicted −
truly +	TP *(true positive)*	FN *(false negative)*
truly −	FP *(false positive)*	TN *(true negative)*

6.2 Five quality measures

Accuracy

proportion of correctly classified objects:

$$\text{ACC} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Easy to read but *misleading on imbalanced data*: predicting always "benign" on a 99%-benign data set gives 99% accuracy and is useless.

Sensitivity
(recall, true positive rate):

$$\text{TPR} = \frac{TP}{TP + FN}.$$

"Of all real positives, how many did we catch?" In medicine: how many sick people did we correctly diagnose.

Specificity
(true negative rate):

$$\text{TNR} = \frac{TN}{TN + FP}.$$

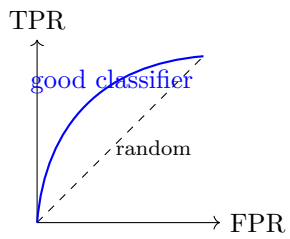
"Of all real negatives, how many did we keep clean?" In medicine: how many healthy people we correctly cleared.

F-measure
(F1, harmonic mean of precision and recall):

$$\text{F}_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}, \qquad \text{precision} = \frac{TP}{TP + FP}.$$

Useful when we care about the positive class only and classes are unbalanced.

AUC
(Area Under the ROC Curve). The ROC curve plots TPR (sensitivity) against FPR (= 1−specificity) as the decision threshold sweeps. AUC is the probability that a random positive scores higher than a random negative. AUC = 1 is perfect, 0.5 is a coin flip.



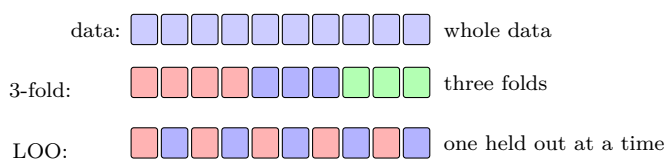
6.3 Four evaluation methods

1. **Resubstitution.** Train on the whole set, evaluate on the same set. *Optimistic* – it tells us how well the model memorises, not how well it generalises. With $k = 1$ resubstitution accuracy is always 100% by construction.

2. **Train/test split.** Randomly split the data, e.g. 70%/30%. Train on the training half, evaluate on the held-out test half. Easy and fast; result depends on the random seed.

3. **M-fold cross-validation.** Cut the data into M folds. Train on $M-1$ of them, test on the remaining fold; rotate; average. $M = 10$ is a popular default. Much more stable than a single split.

4. **Leave-one-out (LOO)** – the extreme case $M = n$. Each object is the test set in turn. Almost no bias but expensive when n is large; with $n = 10$ Dane1 it is trivial, with $n = 569$ in Exercise 2 it is still fast enough.



7 Exercise 2 – k-NN on a real data set

The course says "selected data" – we choose the well-known **Wisconsin Breast Cancer Diagnostic** data set (built into scikit-learn). It has 569 patients, 30 features (radius, texture, perimeter, etc.), and two classes: malignant (212 samples, label 0) and benign (357 samples, label 1). It is a standard benchmark and has the right size: large enough for honest CV/LOO, small enough to run on any laptop.

Pre-processing. All 30 features are continuous but on different scales (radius is around 10–30, area is around 100–2500). Therefore we standardise (`StandardScaler`) before k-NN. We treat *benign* (label 1) as the positive class for sensitivity / F1 / AUC.

7.1 Results – Part I (tables)

We tested $k \in \{1, 3, 5, 7\}$ with all four evaluation methods.

k	evaluation	accuracy	sensitivity	specificity	F1	AUC
1	resubstitution	1.0000	1.0000	1.0000	1.0000	1.0000
1	train/test 70/30	0.9591	0.9626	0.9531	0.9671	0.9579
1	10-fold CV	0.9490	0.9636	0.9245	0.9596	0.9441
1	leave-one-out	0.9508	0.9664	0.9245	0.9610	0.9455
3	resubstitution	0.9807	0.9972	0.9528	0.9848	0.9985
3	train/test 70/30	0.9532	1.0000	0.8750	0.9640	0.9840
3	10-fold CV	0.9666	0.9916	0.9245	0.9739	0.9843
3	leave-one-out	0.9649	0.9916	0.9198	0.9725	0.9820
5	resubstitution	0.9807	0.9972	0.9528	0.9848	0.9983
5	train/test 70/30	**0.9708**	1.0000	0.9219	0.9772	0.9855
5	10-fold CV	0.9701	0.9944	0.9292	0.9766	0.9862
5	leave-one-out	0.9701	0.9916	0.9340	0.9766	**0.9868**
7	resubstitution	0.9719	0.9944	0.9340	0.9780	0.9977
7	train/test 70/30	0.9649	0.9907	0.9219	0.9725	0.9894
7	10-fold CV	0.9649	0.9916	0.9198	0.9725	0.9858
7	leave-one-out	0.9666	0.9916	0.9245	0.9739	0.9881

7.2 Confusion matrix for the best result

The literally highest accuracy in the table above is $k = 1$ resubstitution $= 1.000$. **This is misleading** – it is the textbook overfitting trap of 1-NN: every training point's nearest neighbour is itself. We therefore report the confusion matrix for the best *generalisation* result, which is $k = 5$, 70/30 train/test split (accuracy 0.9708). The cross-validated and leave-one-out results at $k = 5$ are extremely close (0.9701), confirming that the split result is not a lucky outlier.

$k = 5$, 70/30 train/test split (test set: 171 samples):

	predicted benign	predicted malignant
truly benign	107 (TP)	0 (FN)
truly malignant	5 (FP)	59 (TN)

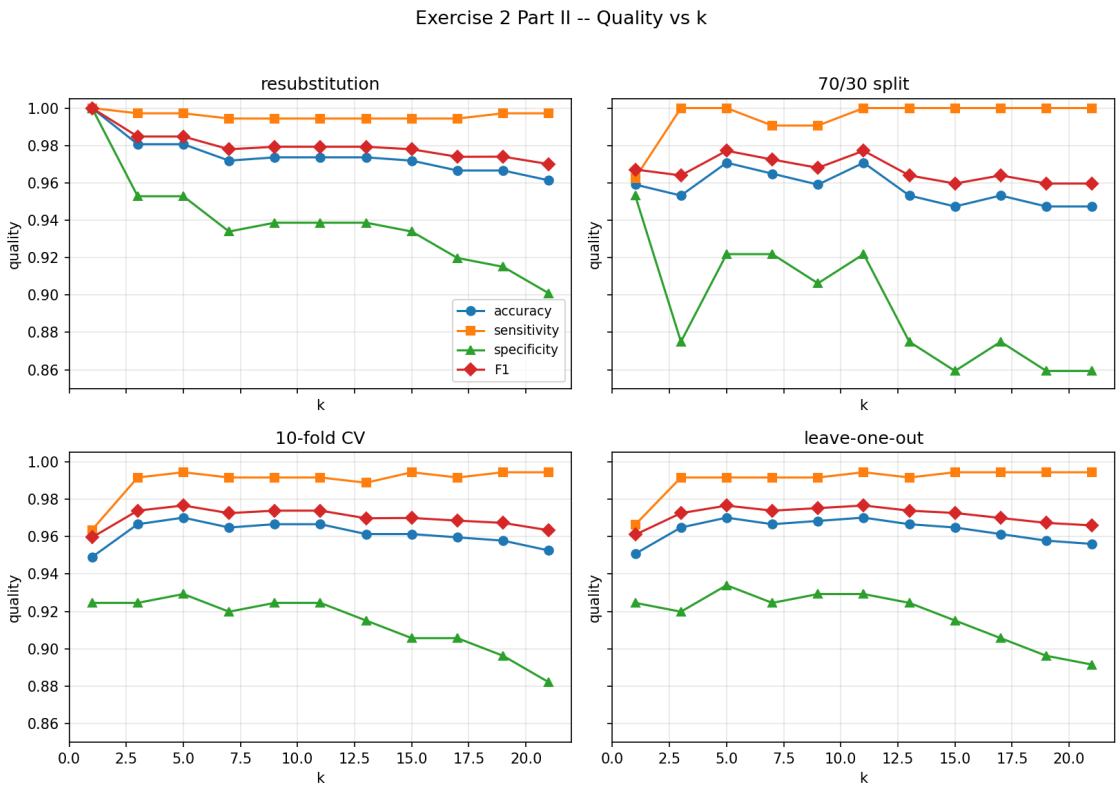
What this means. Out of 107 benign cases in the test set, all 107 were correctly labelled (sensitivity = 107/107 = 1.0). Out of 64 malignant cases, 59 were correctly labelled and 5 were "false alarms" (specificity = 59/64 = 0.9219). In a clinical context the 5 errors here would mean 5 patients sent home with a malignant tumour mis-flagged as benign – that is the kind of error you must minimise by tuning k, by re-weighting classes, or by lowering the decision threshold (which is exactly what AUC summarises across all thresholds).

7.3 Detailed interpretation

- **Resubstitution is dishonest.** Compare $k = 1$ resub (100%) with $k = 1$ LOO (95%). The resubstitution score is the best possible by construction and tells you nothing about future performance.
- **Sensitivity > specificity throughout.** Because benign is the majority class, k-NN naturally tilts in its favour. If we cared more about catching malignant cases, we would either re-sample the data or use weighted voting.
- $k = 5$ **is the sweet spot.** $k = 1$ overfits, $k = 7$ over-smooths. The best *honest* accuracy (CV / LOO) is around 97% at $k = 5$, AUC ≈ 0.987.
- **LOO and 10-fold give almost the same answer** (both 0.9701 for $k = 5$). LOO is unbiased but has higher variance and is more expensive. Ten-fold CV is the practical default.
- **The single train/test split gives a slightly more optimistic 0.9708** only because the random seed happened to be favourable. If you change the seed, you'd see this value bounce around. *Cross-validation is always preferable when you can afford it.*

7.4 Part II – Quality vs. k graphs

The Python script in Section 8 produces the following graph (file `ex2_quality_vs_k.png`):



The curves show that all four metrics behave consistently with the table above: accuracy peaks around $k = 5$, drops slightly at $k = 1$ (overfitting visible only when we evaluate honestly!), and decreases past $k = 15$ as the model becomes too smooth.

8 Python – step by step

8.1 Setting up Python on your own machine

1. **Install Python.** Download the official installer from python.org (any version ≥ 3.10 works). Tick the box "Add Python to PATH" on Windows.

2. **Open a terminal** (`cmd` on Windows, `Terminal` on macOS/Linux) and check:

```
python --version
pip --version
```

3. **Create a project folder** and a virtual environment (recommended – it isolates your packages):

```
mkdir knn_project
cd knn_project
python -m venv .venv
# activate it:
#   Windows:     .venv\Scripts\activate
#   macOS/Linux: source .venv/bin/activate
```

4. **Install the libraries we need:**

```
pip install numpy pandas matplotlib scikit-learn scipy
```

5. **Pick an editor.** VS Code, PyCharm, or even just a Jupyter notebook (`pip install jupyterlab`, then `jupyter lab`). For the script below, save it as `knn_full.py` and run `python knn_full.py`.

A 60-second mental model of the libraries

NumPy – numerical arrays, linear algebra (`np.array`, `np.cov`). **pandas** – spreadsheet-like tables (`DataFrame`) for human-readable results. **matplotlib** – plotting. **scikit-learn** – ready-made machine learning algorithms (`KNeighborsClassifier`, evaluation tools). **SciPy** – scientific helpers (`mahalanobis` distance).

8.2 The complete script

The script `knn_full.py` (in the same folder as this PDF) does *everything* this document describes. The key blocks are:

```python
import numpy as np
from scipy.spatial.distance import mahalanobis

def euclidean(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

def manhattan(a, b):
    return np.sum(np.abs(a - b))

def chebyshev(a, b):
    return np.max(np.abs(a - b))
```

```python

def make_mahalanobis(X):
    cov = np.cov(X, rowvar=False)
    inv = np.linalg.inv(cov)
    return lambda a, b: mahalanobis(a, b, inv)
```

Listing 1: Hand-built distance functions used in Exercise 1

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import (train_test_split, StratifiedKFold,
                                     LeaveOneOut, cross_val_predict)
from sklearn.metrics import (accuracy_score, recall_score, f1_score,
                             roc_auc_score, confusion_matrix)
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True)
X = StandardScaler().fit_transform(X)

def evaluate(y_true, y_pred, y_proba):
    return dict(
        accuracy    = accuracy_score(y_true, y_pred),
        sensitivity= recall_score(y_true, y_pred, pos_label=1),
        specificity= recall_score(y_true, y_pred, pos_label=0),
        f1          = f1_score(y_true, y_pred, pos_label=1),
        auc         = roc_auc_score(y_true, y_proba),
    )

for k in [1, 3, 5, 7]:
    clf = KNeighborsClassifier(n_neighbors=k)
    # Resubstitution
    clf.fit(X, y)
    y_pred  = clf.predict(X)
    y_proba = clf.predict_proba(X)[:, 1]
    print("RESUB", k, evaluate(y, y_pred, y_proba))
    # 10-fold CV
    skf = StratifiedKFold(10, shuffle=True, random_state=42)
    y_pred  = cross_val_predict(KNeighborsClassifier(k), X, y, cv=skf)
    y_proba = cross_val_predict(KNeighborsClassifier(k), X, y, cv=skf,
                                method="predict_proba")[:, 1]
    print("CV10 ", k, evaluate(y, y_pred, y_proba))
```

Listing 2: Exercise 2: evaluation pipeline

The full script (`knn_full.py`) in this folder also produces:

- `ex1_dane1_scatter.png` – scatter plot of Dane1 with the query point and its $k = 5$ neighbours.
- `ex2_quality_vs_k.png` – four lines (accuracy, sensitivity, specificity, F1) versus k for $k = 1, \ldots, 21$, one panel per evaluation method.
- `ex2_confusion_best.png` – visual confusion matrix for the best result.
- `ex2_roc.png` – ROC curves for $k = 5$ across evaluation methods.

8.3 Reading the script the first time

Open `knn_full.py` in your editor and run it. Then play:

- Change `NEW_OBJ = (5.0, 5.0)` at the top of the script to another point and re-run; observe how the prediction changes.

- Change the random seed in `train_test_split(..., random_state=42)`; observe that the test accuracy moves by a few percent. This is exactly why we prefer cross-validation.
- Replace the data set: e.g. `from sklearn.datasets import load_iris; X, y = load_iris(return_X_y=`
  – the rest of the script still runs because every step uses generic feature/label arrays.

8.4 How to apply this to *your* real data

Suppose you have a CSV file called `patients.csv` with one row per patient, several feature columns and one column `diagnosis` taking values 0 or 1.

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("patients.csv")

# 1. Inspect (always do this first!)
print(df.head())
print(df.describe())          # descriptive statistics
print(df["diagnosis"].value_counts())  # class balance

# 2. Separate features X and label y
y = df["diagnosis"].values
X = df.drop(columns=["diagnosis"]).values

# 3. Drop / impute missing values
import numpy as np
mask = ~np.isnan(X).any(axis=1)
X, y = X[mask], y[mask]

# 4. Scale
X = StandardScaler().fit_transform(X)

# 5. Run k-NN exactly like in the script above.
```

Listing 3: Adapting the script to a real CSV

Common mistakes to avoid

1. *Forgetting to scale features.* A single large-scale feature dominates the distance and the algorithm becomes useless.
2. *Evaluating on the training set.* Always hold out something (CV is best).
3. *Using accuracy on imbalanced data.* If 99% of your patients are healthy, "always healthy" wins on accuracy. Look at sensitivity, F1, AUC.
4. *Choosing k by eye.* Use a quality-vs-k curve on the validation data.
5. *Trusting a single train/test split.* The number you see depends on the random seed.

9 What to learn next

- **Other classifiers.** Decision trees, naive Bayes, logistic regression, SVM, random forests, neural networks. Each is a different "shape" of decision boundary; k-NN gives the most local one.
- **Feature engineering and selection.** Garbage in, garbage out. PCA, mutual-information scoring, Lasso.

- **Hyperparameter search.** Grid-search over k and the metric using `GridSearchCV`.
- **Class imbalance handling.** `class_weight=`, SMOTE, threshold tuning.
- **Statistical underpinnings.** Bias-variance trade-off, learning curves, the no-free-lunch theorem.
- **Production considerations.** k-NN is $O(n)$ at prediction time. For large n, look at approximate nearest-neighbour libraries (Faiss, Annoy).

A 30-second study plan

Day 1: re-derive Exercise 1 by hand on paper. Day 2: change the query point, re-run the script, compare. Day 3: replace Dane1 with a tiny CSV of your own. Day 4: switch the data set in Exercise 2 to `load_iris` and verify that the same code still produces sensible numbers. Day 5: read the scikit-learn user guide on classification metrics and on `Pipeline`. *Now you can confidently apply k-NN to a real problem.*

Appendix A. Reading list

- Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*, ch. 13 (free PDF online).
- James, Witten, Hastie, Tibshirani, *An Introduction to Statistical Learning*, ch. 2 & 4.
- scikit-learn user guide: https://scikit-learn.org/stable/modules/neighbors.html.
- Wikipedia article "Mahalanobis distance" for a clean derivation.